PathWise

Software Design Specification

April 2026

Student ID	Student Name & Surname
150122050	Ömer Ercan DAYAN
150123025	Furkan Eren GÜLÇAY
150123028	Mustafa Haktan YAKUT

Prepared for

CSE3044 Software Engineering Term Project

Table of Contents

Table of Contents 2
Document Revision History 3
1. Introduction 4
1.1. Purpose 4
1.2. Statement of Scope 4
1.3. Software Context 4
1.4. Major Constraints 4
1.5. Definitions 5
1.6. Acronyms and Abbreviations 5
1.7. References 6
2. Design Consideration 7
2.1. Design Assumptions and Dependencies 7
2.2. General Constraints 7
2.3. System Environment 7
2.4. Development Methods 8
3. Architectural and Component-Level Design 9
3.1. System Structure 9
3.1.1. Architecture Diagram 9
3.2. Component Descriptions 10
3.2.1. Component: AuthManager (Security & Access Module) 10
3.2.1.1. Processing Narrative (PSPEC) 10
3.2.1.2. Interface Description 10
3.2.1.3. Processing Detail 10
3.2.2. Component: RecommendationEngine (Algorithm Module) 11
3.2.2.1. Processing Narrative (PSPEC) 11
3.2.2.2. Interface Description 11
3.2.2.3. Processing Detail 11
3.2.3. Component: VenueController (Venue Management Module) 12
3.2.3.1. Processing Narrative (PSPEC) 12
3.2.3.2. Interface Description 12
3.2.3.3. Processing Detail 13
3.2.4. Component: UserAccountController (User Management Module) 15
3.2.4.1. Processing Narrative (PSPEC) 15
3.2.4.2. Interface Description 15
3.2.4.3. Processing Detail 15

3.2.5. Component: DatabaseManager / Repository (Data Access Module) 17
3.2.5.1. Processing Narrative (PSPEC) 17
3.2.5.2. Interface Description 17
3.2.5.3. Processing Detail 17
3.2.6. Component: NotificationManager (Organization Module) 18
3.2.6.1. Processing Narrative (PSPEC) 18
3.2.6.2. Interface Description 18
3.2.6.3. Processing Detail 19
3.2.7. Component: PageController (Organization Module) 19
3.2.7.1. Processing Narrative (PSPEC) 19
3.2.7.2. Interface Description 20
3.2.7.3. Processing Detail 20
3.3. Data Design 21
3.4. Dynamic Behavior 21
3.4.1. Interaction Diagrams 21
4. Restrictions, Limitations, and Constraints 24
4.1. Architectural and Technical Constraints 24
4.2. Functional and Business Rules 24
4.3. Data Integrity and Performance 24
5. Conclusion 25

Document Revision History

Date	Version	Description	Author
April 2026	v1.0	Initial complete version	PathWise project team

1. Introduction

This document explains the technical design of the PathWise Travel Recommendation System. It takes the rules we set in the Software Requirements Specification (SRS) and turns them into a practical plan that our team can actually code. Here, we break down how the system is structured, how the frontend and backend talk to each other, and how we handle the database.

1.1. Purpose

The main goal of this document is to serve as a map for our development team. It shows exactly how we will build PathWise so that it meets all the requirements from the SRS. We wrote this for our developers to follow while coding, and for our instructors to evaluate our software architecture.

1.2. Statement of Scope

PathWise is a web application that creates custom travel plans for users based on their budget, destination, and preferences. Here is what we are building, prioritized by importance:

- **Essential:** Secure logins for different user roles (Standard, Business, Admin). A tag-based engine that suggests up to three alternative trips. Screens for venue owners to manage their locations and admins to approve them.
- **Desirable:** Letting users swap out a venue they don't like in their plan. Allowing users to rate suggestions and reset their preference tags.
- **Future:** Real-time hotel prices via external APIs, real AI algorithms, and launching the app on a cloud server

1.3. Software Context

Instead of being a giant search engine like Booking.com that just lists hotels, PathWise acts like a personal travel agent. It uses a tag-matching system to suggest a complete, structured plan based on what the user actually likes.

We are building this using a client-server model. The user interface is written in React, which communicates with our ASP.NET Core backend using REST APIs. All our data is saved securely in a PostgreSQL database using Entity Framework Core. Everything runs locally on our machines for this course project.

1.4. Major Constraints

We have a few hard limits for this project design:

- **Time & Environment:** We only have one academic semester, so we are running everything locally. We will not use cloud infrastructure.
- **No Real AI or Live Payments:** We use a logic-based tag system, not machine learning models. We also won't handle live hotel bookings or external API integrations.
- **Testing & Hardware:** The app must run smoothly on standard student laptops. Also, every core piece of logic must be modular so we can write unit tests for it easily

.

1.5. Definitions

Term	Definition
Tag	A user-selected or system-inferred attribute representing user preferences, used for matching users with venues.
Venue	A location such as a hotel, restaurant, or attraction with associated category, tags, and pricing attributes.
Recommendation	A generated travel plan consisting of one or more venues, organized according to user preferences and system parameters.
TripPlan	A finalized itinerary linking a specific user to a set of venues over a defined timeline and budget constraint.
ORM	Object-Relational Mapping, a technique that maps database entities to application-level objects.
JWT	JSON Web Token, a compact token used for secure stateless authentication.
RDBMS	Relational Database Management System, structured database engine used for persistent data storage.
API	Application Programming Interface, a set of defined endpoints enabling communication between system components.
SRS	Software Requirements Specification, the document defining all functional and non-functional requirements for PathWise.
DSD	Design Specification Document, this document, describing the architectural and component-level design.

1.6. Acronyms and Abbreviations

Acronym	Expansion
SRS	Software Requirements Specification
DSD	Design Specification Document
API	Application Programming Interface
RDBMS	Relational Database Management System
ORM	Object-Relational Mapping
JWT	JSON Web Token
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer
UI	User Interface
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case

1.7. References

- CSE3044 DSD Template — provided by course instructor.
- PathWise Software Requirements Specification (SRS), v1.0, March 27th, 2026.
- IEEE Guide to Software Requirements Specification (ANSI/IEEE Std. 830-1984).
- Microsoft ASP.NET Core Documentation — https://learn.microsoft.com/en-us/aspnet/core
- React Documentation — https://react.dev
- Entity Framework Core Documentation — https://learn.microsoft.com/en-us/ef/core

2. Design Consideration

This section outlines the environment, rules, and assumptions that shaped our architecture. It explains the practical context and the "why" behind our technical choices.

2.1. Design Assumptions and Dependencies

Before building PathWise, we made a few assumptions about where and how it will run:

- **Environment:** We assume developers and evaluators have a standard machine capable of running a React frontend, an ASP.NET Core backend, and a PostgreSQL database. It should work seamlessly on Windows, Linux, or macOS.
- **User Skills:** We expect Standard Users to know how to navigate basic web forms. Business Owners need slightly more digital literacy to manage their venues, while Administrators should be comfortable handling system data and moderation panels.
- **Future-Proofing:** Even though we are not doing it now, we assume the system might eventually need cloud hosting, live pricing APIs, or machine learning. We designed the architecture to be flexible enough to allow these upgrades later

2.2. General Constraints

These are the hard technical rules our team must follow during development:

- **Hardware Limits:** Since we are not using cloud servers, the system must run smoothly on a standard student laptop (minimum 4 GB RAM, dual-core CPU) without crashing.
- **Security & Standards:** All communication between the frontend and backend must be RESTful. Passwords must be securely hashed using BCrypt. User sessions (JWT) will expire after 15 minutes of inactivity, and accounts will automatically lock out after 5 failed login attempts to prevent brute-force attacks.
- **Performance Targets:** Everyday actions like logging in or viewing a profile must take under 2 seconds. The heavy lifting—generating travel plans or calculating venue alternatives—must finish within 3 seconds.
- **Testing:** We must write highly modular code so we can easily isolate and unit test our backend logic using xUnit or NUnit.

2.3. System Environment

This is the core technology stack we are using to build and test PathWise:

Category	Tool / Technology	Purpose
Frontend	React (JavaScript)	User interface layer
Backend	ASP.NET Core Web API (C#)	Business logic and REST API
Database	PostgreSQL (RDBMS)	Persistent data storage

ORM	Entity Framework Core	Database abstraction layer
Auth	JWT (JSON Web Token)	Session and token management
IDE	Visual Studio / VS Code	Development environment
Version Control	Git / GitHub	Source code management
Testing	xUnit / NUnit	Backend unit testing
Browser	Chrome, Firefox, Edge	Frontend testing and usage

2.4. Development Methods

We are building PathWise using a **Layered Architecture**. This means we physically separate the code into distinct tiers: the UI (React), the API (ASP.NET Core), the Business Logic (Recommendation Engine), and Data Access (EF Core + PostgreSQL).

To keep our codebase clean and maintainable, we rely on core software engineering principles:

- **SOLID & GRASP:** We assign specific responsibilities to specific classes (for example, using Controllers just for routing and Services for business logic). This prevents our code from becoming tightly coupled or messy.
- **Separation of Concerns:** Each module minds its own business. For example, the Venue Management module doesn't worry about how user passwords are saved.
- **Iterative Process:** We are building the essential features first (like secure logins and basic plan generation) before moving on to the "nice-to-have" features

3. Architectural and Component-Level Design

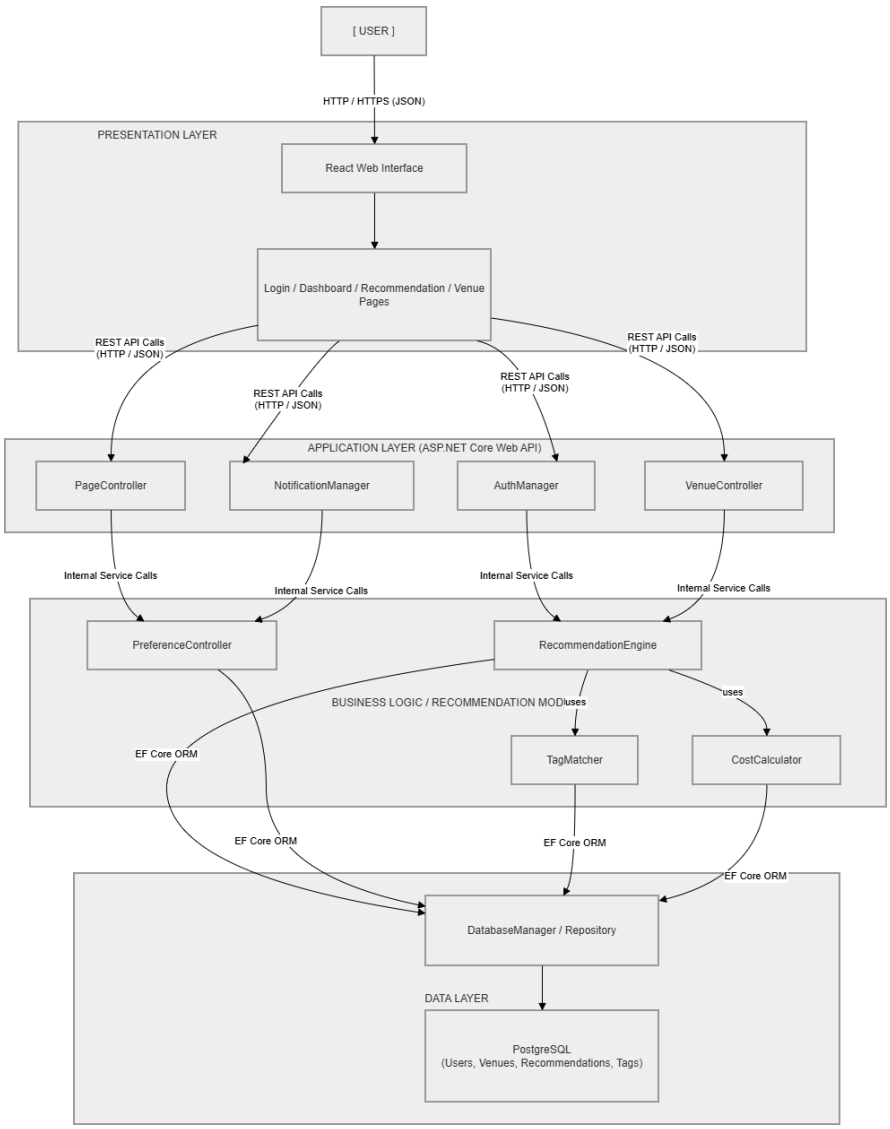
This section provides a detailed description of the PathWise system architecture, its major components, their responsibilities, interfaces, and processing behavior.

3.1. System Structure

PathWise adopts a three-tier client-server architecture consisting of a Presentation Layer, an Application and Business Logic Layer, and a Data Layer. Each tier is independently deployable and communicates with adjacent tiers through well-defined interfaces.

3.1.1. Architecture Diagram

The following diagram illustrates the high-level layered architecture of the PathWise system:



The layered architecture ensures that each tier only communicates with the layer directly adjacent to it, enforcing separation of concerns and facilitating independent testing and maintenance.

3.2. Component Descriptions

3.2.1. Component: AuthManager (Security & Access Module)

3.2.1.1. Processing Narrative (PSPEC)

The AuthManager is responsible for all authentication and authorization operations within the PathWise system. It validates user credentials, generates and validates JWT session tokens, enforces role-based access control (RBAC), and manages account security policies including failed-login lockout and session expiry. Every API request that requires authentication passes through the AuthManager before reaching business logic components.

3.2.1.2. Interface Description

Direction	Parameter	Type	Description
Input	email	string	Registered user e-mail address
Input	password	string	Plain-text password (hashed before use)
Output	JWT Token	string	Signed token containing user ID and role
Output	Role	enum	StandardUser | BusinessOwner | Admin
Output	Error Message	string	Descriptive failure message on error

3.2.1.3. Processing Detail

- Receive login request with email and password from the API controller.
- Validate input format (email RFC 5322, password non-empty).
- Check failed-login counter; if >= 5, reject with lockout error.
- Query DatabaseManager for user record matching the provided email.
- Hash the input password with BCrypt and compare to stored hash.
- If match: generate JWT token with user ID, role, and expiration (15-minute inactivity timeout).
- If no match: increment failed-login counter and return authentication error.
- For Business Owner accounts in 'Pending' status: return account-pending error.
- Return token and role to API controller for session creation.

Design Class Hierarchy:

- AuthManager (controller class)
- → uses: DatabaseManager/Repository (data access)
- → uses: BCryptHasher (password hashing utility)
- → produces: JwtTokenService (token generation and validation)

Restrictions / Limitations:

- Passwords must never be stored or transmitted in plain text.
- Tokens must be validated on every protected API endpoint before processing.
- Account lockout lasts 1 minute after 5 consecutive failed login attempts.

Performance Issues:

- The entire authentication process must complete within 2 seconds under normal conditions.
- BCrypt hashing introduces minor latency (~100–300ms); this is acceptable for security purposes.

Design Constraints:

- Authentication uses JWT with a standard claims structure (sub, role, exp).
- The AuthManager must be stateless; session state is stored in the token, not the server.

3.2.2. Component: RecommendationEngine (Algorithm Module)

3.2.2.1. Processing Narrative (PSPEC)

The RecommendationEngine is the core algorithmic component of PathWise. It receives user travel parameters and generates up to three personalized alternative travel plans (TripPlans). The engine performs venue retrieval, preference-based filtering, similarity scoring, chronological scheduling, and cost estimation. It interacts with the DatabaseManager to retrieve venue and user preference data, and with the TagMatcher utility to compute similarity scores.

3.2.2.2. Interface Description

Direction	Parameter	Type	Description
Input	destination	string	Target city or region for the trip
Input	startDate / endDate	DateTime	Travel date range
Input	budget	decimal	Maximum spending limit per plan
Input	travelerCount	int	Number of travelers
Input	userTags	List<Tag>	User's selected preference tags
Output	TripPlans	List<TripPlan>	Up to 3 ranked travel plans
Output	Error Message	string	Returned when no venues found or inputs invalid

3.2.2.3. Processing Detail

- Validate all input parameters (non-null, date consistency, budget > 0).
- Query DatabaseManager to retrieve all Approved venues in the specified destination.
- Filter venues by budget: remove venues where cost × travelerCount > budget.
- Filter venues by capacity: remove venues where capacity < travelerCount.
- If user has historic preference data, retrieve stored tag weights from the database.

- Apply TagMatcher to compute similarity scores between venue tags and user preference tags (weighted by historical data if available).
- Rank candidate venues by similarity score (descending).
- Construct up to three TripPlans by selecting top-ranked venue combinations, scheduling them chronologically within the travel date range, and ensuring no date overlaps.
- For each TripPlan, calculate total estimated cost via CostCalculator.
- Return the ranked list of TripPlans.

Design Class Hierarchy:

- RecommendationEngine (core algorithm class)
- → uses: DatabaseManager (venue and user data retrieval)
- → uses: TagMatcher (similarity scoring utility)
- → uses: CostCalculator (total plan cost estimation)
- → produces: TripPlan (domain entity)

Restrictions / Limitations:

- Only administrator-approved venues may appear in recommendations.
- Recommended venues must be within the specified destination; no cross-destination mixing.
- Generated plans must not exceed the user's budget.
- If fewer than three valid plans can be constructed, the system returns however many are available.

Performance Issues:

- Plan generation must complete within 3 seconds. To achieve this, venue filtering is performed before scoring to minimize computation.
- Tag comparison uses a precomputed tag vector; complex ML models are excluded from scope.

Design Constraints:

- The engine is stateless — all required data is retrieved fresh per request.
- Recommendation quality is constrained by the completeness and accuracy of tags assigned to venues.

3.2.3. Component: VenueController (Venue Management Module)

3.2.3.1. Processing Narrative (PSPEC)

The VenueController manages the full lifecycle of venue data in the PathWise system. It provides CRUD operations (Create, Read, Update, Delete) for Business Owners, enforces the approval workflow (new and updated venues are marked as 'Pending' until an Administrator approves them), and supports the Administrator's moderation operations (approve or reject submissions). It ensures that only approved venues are visible to Standard Users and the RecommendationEngine.

3.2.3.2. Interface Description

Direction	*Parameter*	*Type*	*Description*

Input	createRequest	VenueCreateDto	Contains the data required to create a new venue, including venue details, category, pricing, capacity, and tag associations.
Input	updateRequest	VenueUpdateDto	Contains the editable venue information used when updating an existing venue record.
Input	venueId	Guid / int	Unique identifier of the venue to be retrieved, updated, or deleted.
Input	ownerId	Guid / int	Identifier of the authenticated business owner performing the operation.
Input	locationFilter	string	Optional location-based filter used when listing approved venues for standard users.
Output	venueDetails	VenueDetailsDto	Detailed representation of a single venue returned after create, update, or retrieval operations.
Output	venueList	List<VenueDetailsDto>	Collection of venue records returned for listing operations.
Output	status	VenueStatus	Indicates the moderation state of the venue such as Pending, Approved, or Rejected.
Output	resultMessage	string	Operation result message indicating success, failure, or validation/moderation feedback.

3.2.3.3. Processing Detail

Create Venue:

- Validate all required fields (name, category, location, price range, capacity, tags ≥ 3).
- Confirm that the requesting user has Business Owner role.
- Create a new Venue record with status = 'Pending'.
- Persist to database via DatabaseManager.
- Notify Administrator of new pending submission via NotificationManager.
- Return confirmation message and updated venue list.

Update Venue:

- Verify that the requesting Business Owner owns the venue (ownerID match).
- Validate updated fields.
- Reset venue status to 'Pending' after update.
- Persist changes and notify Administrator.

Delete Venue:

- Verify ownership.
- Remove venue record from database.
- Return confirmation message.

Moderation (Admin Approve/Reject):

- Verify Administrator role.
- Retrieve venue details for review.
- On approve: update status to 'Approved', venue becomes available in system.
- On reject: update status to 'Rejected', venue remains hidden.
- Record moderation action and timestamp in database.

List Venues:

- The system checks the role of the user who sent the request.
- If the user is a "Business Owner," the system retrieves all venues associated with that ownerID from the database (regardless of whether they are Pending, Approved, or Rejected).
- If the user is a "Standard User," only venues with a status of 'Approved' that match the selected location are retrieved from the database.

Restrictions / Limitations:

- Only Business Owners can submit or modify venues; only Admins can approve or reject.
- All venue operations must be performed within authenticated sessions.
- A minimum of 3 tags must be assigned to each venue.
- Rejected venues are never visible to Standard Users.

Decision Log:

- The VenueController was kept focused on orchestration responsibilities, while venue creation, update, retrieval, and lifecycle rules were delegated to IVenueService in order to preserve separation of concerns.
- A service abstraction was preferred over direct controller-to-data access so that the module remains testable, loosely coupled, and aligned with dependency inversion principles.
- Venue-related validation was separated into dedicated request validators, allowing business rules and input-quality checks to evolve independently without making the controller overly procedural.
- DTOs were used as the boundary models for venue operations so that the internal venue entity can remain protected from direct external exposure and unnecessary coupling.
- Venue persistence was intentionally routed through UnitOfWork and repository abstractions, which supports controlled data access and keeps the controller and service layers independent from storage details.
- Tag handling was treated as an explicit part of venue management because recommendation quality depends on structured venue metadata, making tag associations an important design concern rather than a minor attribute update.
- A notification dependency was included so that venue submission and approval-related workflows can interact naturally with moderation processes without embedding those concerns directly into the controller.
- Mapping support was adopted to keep transformations between venue entities and response models consistent, reducing repeated conversion logic and making the service layer easier to maintain.

- Venue status transitions such as pending, approved, and rejected were modeled as part of the component workflow so that moderation requirements could be reflected directly in the module design.

3.2.4. Component: UserAccountController (User Management Module)

3.2.4.1. Processing Narrative (PSPEC)

The UserAccountController handles user registration, account deletion, and preference profile management. It manages the full lifecycle of a Standard User account, from initial registration (including first-time tag selection) through account deletion. It also supports Business Owner registration, which requires Administrator approval before activation.

3.2.4.2. Interface Description

Direction	*Parameter*	*Type*	*Description*
Input	registerRequest	RegisterRequestDto	Contains the information required for account creation, such as identity, credentials, account type, and initial registration data.
Input	profileRequest	UserProfileDto	Contains the editable user profile data used for profile update operations.
Input	userId	Guid / int	Unique identifier of the target user account.
Input	preferenceTagIds	List<Guid> / List<int>	Collection of selected preference tag identifiers used for profile initialization or reset-related operations.
Input	deletionConfirmation	bool	Indicates whether the user has explicitly confirmed the account deletion action.
Output	userDetails	UserDetailsDto	Detailed representation of the user account returned after read or update operations.
Output	accountStatus	AccountStatus / string	Indicates the current user account state such as Active, Pending, or Deleted.
Output	resultMessage	string	Operation result message describing success, failure, approval state, or deletion outcome.

3.2.4.3. Processing Detail

Registration:

- Validate all input fields (RFC 5322 email, password strength policy, non-empty name).
- Check if email already exists in the database.
- Hash password using BCrypt.
- For Standard User: create account with status = 'Active'; redirect to tag selection page.
- For Business Owner: create account with status = 'Pending Approval'; forward request to Admin dashboard.

- Notify Administrator of new Business Owner registration (via NotificationManager).

Account Deletion:

- Verify active authenticated session.
- Display confirmation prompt; user must explicitly confirm.
- Atomically delete all user data (account, preference profile, recommendation history).
- Terminate session immediately.
- Redirect to login page with deletion confirmation message.

Preference Profile Reset:

- Verify user is authenticated.
- Check if preference profile exists.
- Clear all stored tag weights and historical preference data.
- Reinitialize preference state to 'Empty'.
- Notify RecommendationEngine of reset state.
- Confirm reset to user.

Update Preference Profile:

- It is verified that the user has an active session.
- The user sends a new tagList
- The system updates the old label weights in the database with the newly selected ones.

Decision Log:

- The UserAccountController was designed as a coordination layer that delegates account-related operations to IUserService, keeping user lifecycle rules separate from interface-level concerns and preserving a clear layered structure.
- An interface-based service dependency was preferred over direct concrete service usage so that the module remains loosely coupled, easier to test, and more adaptable to future implementation changes.
- Validation responsibilities were separated from business operations through dedicated validators and request models, allowing user-related input rules to evolve without overloading the controller or service layer.
- DTO-based communication was adopted instead of exposing domain entities directly, which helps preserve encapsulation, stabilize external contracts, and prevent internal model leakage across layers.
- Preference-related behavior was treated as a collaborating concern rather than being embedded directly into controller logic, allowing user profile management and preference reset operations to remain modular and easier to maintain.
- Notification support was included as a secondary dependency so that account-related workflows, especially those involving approvals or user-state changes, can trigger follow-up actions without tightly coupling the module to messaging details.
- User persistence was intentionally routed through UnitOfWork and repository abstractions, ensuring that account updates, deletions, and related preference cleanup remain centralized and transaction-aware.

- Mapping utilities were introduced to standardize transformations between domain models and user-facing data structures, reducing repetitive conversion logic and improving consistency across the module.

3.2.5. Component: DatabaseManager / Repository (Data Access Module)

3.2.5.1. Processing Narrative (PSPEC)

The DatabaseManager serves as the data access abstraction layer for the entire PathWise system. It encapsulates all database interactions including queries, inserts, updates, and deletes through the Entity Framework Core ORM. No other component directly interacts with the database; all data operations are routed through the DatabaseManager. This ensures data integrity and simplifies maintenance of the persistence layer.

3.2.5.2. Interface Description

Operation	*Input*	*Output*
GetUserByEmail	email: string	User? (null if not found)
SaveUser	user: User	bool (operation result)
GetVenuesByLocation	location: string, status: VenueStatus	List<Venue>
SaveVenue	venue: Venue	bool (operation result)
UpdateVenue	venue: Venue	bool (operation result)
GetUserPreferences	userId: Guid / int	List<UserPreference>
SaveRecommendation	tripPlan: TripPlan	bool (operation result)
DeleteUser	userId: Guid / int	bool (operation result)
UpdateVenueStatus	venueId: Guid / int, status: VenueStatus	bool (operation result)

3.2.5.3. Processing Detail

- All database operations are executed within transactions to guarantee atomicity.
- Referential integrity is enforced through foreign key constraints (e.g., Venue.ownerID → User.userID).
- Entity Framework Core handles query generation and object mapping automatically.
- In case of transaction failure, rollback is applied to prevent partial writes.
- Database connection pooling is used to maintain performance under concurrent requests.

Restrictions / Limitations:

- No other component may directly query the database; all access must go through this module.
- Schema changes require database migration scripts (EF Core Migrations).
- Performance of complex queries may degrade as data volume grows; query optimization may be needed in future iterations.

Decision Log:

- The data access module was structured around the Repository and UnitOfWork patterns so that persistence logic is centralized and other modules do not interact with the database directly.
- A generic repository base was selected for common CRUD functionality in order to reduce duplication, while separate repositories were retained for entity-specific queries and future extensibility.
- UnitOfWork was positioned as the main coordination point of the data layer so that multiple repository operations can be grouped under a single controlled persistence boundary.
- Entity Framework Core was used through ApplicationDbContext as the concrete persistence mechanism, allowing the design to benefit from ORM support while still keeping upper layers insulated through abstractions.
- Separate DbSet<TEntity> definitions were maintained for major domain entities to reflect the structure of the domain model clearly and to provide explicit entry points for repository operations.
- Repository access was deliberately kept behind interfaces to support loose coupling, mocking in tests, and easier replacement of implementation details if the persistence strategy changes later.
- The module was designed to act as a shared infrastructure layer rather than a business decision-maker, ensuring that query execution and data storage concerns remain isolated from application and domain rules.
- This structure was also chosen to improve maintainability over time, since schema-related changes can be localized in the data access layer instead of spreading across controllers and services.

3.2.6. Component: NotificationManager (Organization Module)

3.2.6.1. Processing Narrative (PSPEC)

The NotificationManager is the central communication module responsible for coordinating system-wide alerts. It handles internal events—such as notifying Administrators of new venue submissions, alerting Business Owners of approval decisions, and warning Standard Users about budget limits. Instead of relying on external email services, this module manages notifications as persistent, database-backed alerts that are surfaced directly on the users' role-specific dashboards. It serves as a shared utility accessed by other domain services (e.g., VenueService, UserService) to ensure users are promptly informed of state changes within the system.

3.2.6.2. Interface Description

Direction	*Parameter*	*Type*	*Description*
Input	createRequest	NotificationCreateDto	Contains the data required to generate a new system alert (message content, recipient ID, and type)..
Input	notificationId	Guid / int	Unique identifier used to fetch a specific notification or mark it as read.
Input	userId	Guid / int	Identifier of the user to fetch the targeted notification list.
Output	notificationList	List<NotificationDto>	Collection of notifications returned to be displayed on the user's dashboard.
Output	notificationDetails	NotificationDto	Detailed representation of a single notification object.

Output	resultMessage	String	Operation result indicating whether the notification was successfully processed or delivered.

3.2.6.3. Processing Detail

Notification Workflow:

1. An internal service (e.g., VenueController) or external request initiates an alert by submitting a NotificationCreateDto.
2. The payload is validated using the NotificationCreateValidator to ensure required fields (like recipient ID and message content) are correctly formatted.
3. The NotificationController delegates the validated data to the INotificationService.
4. The service constructs a new Notification entity (setting isRead = false and assigning a timestamp) and persists it to the database via IUnitOfWork and IRepository<Notification>.
5. When a user navigates to their dashboard, a GET request is made to fetch their specific alerts. The service retrieves the records, maps them into NotificationDto objects using IMapper, and returns a clean JSON response.
6. Once the user interacts with an alert, a separate request updates the notification's status to isRead = true, which is then committed to the database.

Decision Log:

- The NotificationManager was designed using a layered approach, separating the HTTP request handling (NotificationController) from the actual notification delivery and state management logic (INotificationService). This ensures the module adheres to the Single Responsibility Principle.
- Validation logic for incoming notification requests was offloaded to a dedicated NotificationCreateValidator (implementing IValidator<T>). This keeps the controller lean and centralizes business rules regarding notification formats.
- Communication with the presentation layer is strictly handled via Data Transfer Objects (NotificationDto, NotificationCreateDto) to prevent exposing the internal database entity and its metadata to the frontend.
- Data persistence is routed through the IUnitOfWork and IRepository<Notification> interfaces. This ensures that marking notifications as read or creating new alerts remains transactionally safe and decoupled from the service logic.

3.2.7. Component: PageController (Organization Module)

3.2.7.1. Processing Narrative (PSPEC)

The PageController functions as a centralized "Dashboard Aggregator" and gateway between the React frontend and the backend service layer. Rather than acting as a universal router for all system operations, its primary responsibility is to compile role-specific summary data (e.g., pending approvals for Administrators, active listings for Business Owners, and recent itineraries for Standard Users) into a single, optimized response. Upon receiving a dashboard load request, it validates the user's session and role, then delegates the complex data gathering to the IPageService. By aggregating read-only metrics across multiple repositories into a unified

DashboardSummaryDto, it significantly reduces frontend-backend network traffic while leaving state-changing operations (Create, Update, Delete) strictly to their respective domain controllers.

3.2.7.2. Interface Description

Direction	*Parameter*	*Type*	*Description*
Input	userId	Guid / int	Identifier of the authenticated user requesting the dashboard data.
Input	userRole	enum / string	The role (Admin, BusinessOwner, StandardUser) used to determine which specific metrics and widgets to aggregate.
Output	dashboardSummary	DashboardSummaryDto	An aggregated data package containing metrics such as active venues, pending approvals, and recent trip plans tailored to the user's role.
Output	resultMessage	string	Operation status message for the page load request.

3.2.7.3. Processing Detail

Dashboard Aggregation Workflow:

1. Receive an HTTP GET request from the frontend for the user's dashboard summary.
2. Extract and validate the userId and userRole claims from the authenticated session token (JWT).
3. Delegate the data aggregation task to the IPageService.
4. IPageService initiates read-only queries via IUnitOfWork across multiple repositories based on the user's role:
    - **For Administrators:** Count pending venue submissions, active users, and system alerts.
    - **For Business Owners:** Retrieve statistics for their owned venues, active listings, and related notifications.
    - **For Standard Users:** Fetch recent travel plans (TripPlans) and active preference tags.
5. Map the retrieved raw data entities into a unified DashboardSummaryDto utilizing IMapper.
6. Return the structured JSON response to the client, ensuring minimal payload size.

General Routing & Validation:

1. Validate the structural integrity of incoming requests using FluentValidation before delegating them to underlying services.
2. Enforce strict Role-Based Access Control (RBAC) at the endpoint level to ensure users cannot access or aggregate data outside their authorized scope..

Decision Log:

- The PageController was conceptualized as a "Dashboard Aggregator" or "Gateway" to optimize communication between the React frontend and the ASP.NET Core backend. Instead of the frontend making multiple separate API calls to fetch user stats, venue counts, and recent plans, the PageController aggregates this into a single response.
- An IPageService abstraction was introduced to handle the complex aggregation logic. This service interacts with the IUnitOfWork to perform read-only queries across multiple repositories (e.g., Venue, TripPlan) without violating the autonomy of other domain controllers.
- The DashboardSummaryDto was created specifically to package these aggregated metrics. This ensures that only the necessary summary data is transmitted, optimizing network bandwidth and adhering to the Information Expert pattern.
- To maintain strict Separation of Concerns, the PageController is restricted to read-only aggregation for dashboard views; any state-changing operations (Create, Update, Delete) are strictly routed to their respective domain controllers (e.g., VenueController, UserAccountController).

3.3. Data Design

The PathWise system uses a relational database with the following core entities and relationships:

Entity	Primary Attributes	Relationships
User	userID (PK), name, email, passwordHash, role, status	1 User → N Recommendations; 1 User → N Venues (if owner)
Venue	venueID (PK), venueName, location, category, priceRange, capacity, status, ownerID (FK)	N Venues → 1 Owner; N Venues ↔ N Tags; N Venues ↔ N Recommendations
Tag	tagID (PK), tagName	N Tags ↔ N Venues; N Tags ↔ N Users (preference profile)
Recommendation	recommendationID (PK), userID (FK), generatedDate, rating	1 Recommendation → N Venues (via join table)
Notification	notificationID (PK), recipientID (FK), message, type, isRead, createdAt	N Notifications → 1 User

3.4. Dynamic Behavior

3.4.1. Interaction Diagrams

The following sequence diagrams summarize the key dynamic behaviors of the PathWise system. Detailed sequence diagrams for all eight use cases (UC#1 through UC#8) are defined in Section 4.3 of the PathWise SRS document (v1.0, March 27, 2026) and are referenced here.

UC#1 — User Registration:

User provides registration data → System validates inputs → For Standard User: account created as Active → First-time login triggers tag selection. For Business Owner: account created as Pending → Admin is notified → Admin approves or rejects → User is notified of outcome.

UC#2 — User Login:
User provides credentials → Frontend sends to AuthManager → AuthManager validates credentials and checks lockout status → On success: JWT token is generated by SessionManager and returned → User is redirected to role-appropriate dashboard. On failure: error message returned; failed-login counter incremented; temporary lockout applied if threshold exceeded.

UC#3 — Travel Plan Recommendation:
User enters travel parameters → Frontend forwards to RecommendationEngine via API → Engine retrieves and filters venues from database → If user has history: historical weights applied; otherwise input tags used → Venues ranked by similarity score → Chronologically organized → Cost estimated → Up to 3 finalized TripPlans returned to user.

UC#4 — Venue Management:
Business Owner submits create/update/delete request → VenueController validates inputs → On create/update: venue saved with 'Pending' status → Admin notified → On delete: venue record removed → Confirmation returned. Admin approval changes status to 'Approved', making venue available system-wide.

UC#5 — Venue Moderation:
Admin opens moderation panel → System retrieves pending venues → Admin selects a venue and reviews details → Admin approves: status updated to 'Approved', venue enters recommendation pool → Admin rejects: status updated to 'Rejected', venue remains hidden → Decision logged in database.

UC#6 — User Account Deletion:
User requests account deletion → System prompts confirmation → User confirms → AccountController atomically deletes all user data (account, preferences, history) → Session terminated immediately → User redirected to login page.

UC#7 — Preference Profile Reset:
User requests preference reset → System prompts confirmation → User confirms → PreferenceController clears all stored tag weights → Profile reinitialized to empty state → RecommendationEngine notified → User informed that future recommendations will treat them as a new user.

UC#8 — Initial Preference Tag Selection:

After first login, system detects uninitialized preference profile → User redirected to tag selection page → System displays available tags → User selects minimum 3 tags and submits → System validates selection → Tags saved to database → RecommendationEngine initialized with user preferences → User redirected to dashboard.

4. Restrictions, Limitations, and Constraints

The following restrictions and design constraints define the boundaries of the PathWise system. These limitations were intentionally set to maintain a focused scope, ensure data security, and adhere to the architectural decisions made during the design phase:

4.1. Architectural and Technical Constraints

- **Strict Layered Communication:** To maintain the Separation of Concerns, controllers (e.g., VenueController, PageController) are strictly prohibited from accessing the database directly. All data operations must pass through their respective Service interfaces and the *IUnitOfWork*.
- **Mandatory DTO Usage:** Internal database entities (like User or Venue) must never be exposed to the React frontend. All data entering or leaving the API must be packaged into Data Transfer Objects (DTOs) to prevent sensitive data leakage and optimize network payload.
- **Local Environment Scope:** The system is designed for a local development and academic demonstration environment. Cloud deployment, load balancing, and external API integrations (such as real-time hotel pricing or booking gateways) are currently out of scope.

4.2. Functional and Business Rules

- **Role-Based Isolation:** A Standard User cannot access Business Owner or Administrator endpoints. This constraint is not just handled on the frontend; it is strictly enforced at the API layer using stateless JWT claims
- **Read-Only Aggregation:** The PageController functions solely as a "Dashboard Aggregator". It is constrained to read-only queries. Any state-changing actions (creating a venue, deleting an account) must be routed to their specific domain controllers
- **Algorithm Limits:** The recommendation engine relies on a deterministic, tag-based weighting system rather than machine learning models. It is restricted to suggesting only "Approved" venues that strictly match the user's requested destination

4.3. Data Integrity and Performance

- **Atomic Transactions:** Any operation that modifies multiple records (e.g., a Business Owner deleting a venue, which also deletes related tags and triggers an Admin notification) must be executed as a single atomic transaction via UnitOfWork. Partial database writes are not acceptable.
- **Response Time Limits:** Because the system operates synchronously, standard operations (login, fetching dashboards) are constrained to complete within 2 seconds. The complex travel plan generation is limited to a maximum of 3 seconds to ensure a smooth user experience.
- **Credential Security:** Plain-text passwords are fundamentally incompatible with the system's security constraints. All credentials must be hashed using BCrypt before reaching the database layer

5. Conclusion

This Software Design Specification (DSD) provides a comprehensive architectural blueprint for the PathWise Travel Recommendation System. Throughout this document, the functional and non-functional requirements established in the SRS have been translated into a concrete, testable, and highly modular technical design.

At its core, PathWise is built upon a modern three-tier architecture—utilizing a React frontend, an ASP.NET Core backend, and a PostgreSQL database. To ensure long-term maintainability and strict adherence to SOLID principles, the system was carefully divided into seven primary modules. By implementing established design patterns such as the Repository Pattern, Unit of Work, and strict DTO-based communication, the architecture successfully isolates complex business logic from data access and presentation layers.

The technical choices highlighted in our Decision Logs reflect a deliberate engineering mindset. Rather than overcomplicating the system with out-of-scope machine learning models or unnecessary external API dependencies, the design focuses on a robust, deterministic tag-based recommendation engine. It efficiently calculates and matches optimal combinations of hotels and venues based on user budgets and locations, all securely governed by stateless JWT authentication and strict Role-Based Access Control (RBAC).

By acknowledging our constraints and clearly defining system boundaries, data flows, and component interactions, we have eliminated architectural ambiguities. The PathWise development team is now fully equipped to transition into the implementation and coding phase, confident that this design provides a secure, scalable, and requirements-compliant foundation